Chapter 12

Generative AI for Recommender Systems

After reading this chapter, you will be able to:

- Place a generative model at each stage of a recommendation funnel (offline enrichment, retrieval, ranking, and re-ranking) and reason about the serving cost and latency that each placement adds.
- Build Semantic IDs with residual quantization, diagnose codebook collapse, collisions, and invalid generations, and explain how generative retrieval decodes valid item identifiers with constrained beam search.
- Explain how a generative recommender such as HSTU turns a user's actions into a token sequence for ranking and retrieval, and how scoring many candidates against one encoded history keeps its serving cost manageable.
- Use a large language model offline to enrich items and generate labels, choose an encoder whose embedding space matches the model that consumes the features, and distil large-model behaviour into a student model that can be served.
- Prompt and fine-tune language models as rankers, mask labels correctly during fine-tuning, and describe where generative reasoning re-rankers fit and which output checks they need.
- Evaluate LLM-based recommenders without being misled by how candidate slates are built, by leakage, or by parsing fallbacks.
- Choose an integration pattern that fits a product's data, latency budget, and team, and plan for hallucinated items, prompt sensitivity, model licences, privacy, cost, and serving-time agents.

12.1 Why Generative AI Changes the Recommender Stack

Consider what happens when a new item enters a catalog. Most production recommenders identify each item by an arbitrary identifier that indexes a row in an embedding table. That row starts as random numbers and becomes useful only after the item has collected enough interactions for training to shape it. Chapter 10 showed how content embeddings give a new item a meaningful vector from its text and images, but swapping item identifiers for content embeddings is not free. When researchers at YouTube replaced the video identifier in a production ranking model with a content embedding, overall quality dropped, because the model could no longer memorize the behaviour of individual popular videos (Singh et al., 2024). Recommenders need both memorization of what each item has done and generalization to items that have done nothing yet.

Two further limits recur throughout this book. Ranking models depend on large sets of hand-built features, such as counts, ratios, and crosses, that teams maintain for years, and Chapter 4 described how much engineering those pipelines absorb. Zhai et al. (2024) observe that such deep learning recommendation models (DLRMs) often stop improving as more compute is added. Models trained only on interaction logs also know nothing that the logs do not show. A model that has never seen a camping

stove bought alongside a gas canister cannot infer that they belong together, while a model that has read product descriptions and reviews can (Xi et al., 2024).

Generative AI addresses each of these limits, and it brings vocabulary that the rest of the chapter uses. A large language model (LLM) is a transformer, the architecture introduced in Chapter 11, trained on very large text corpora to predict the next token. A token is the unit of input that the model reads and writes. For text, a token is a word or a fragment of a word; in this chapter, tokens will also stand for item codes and for user actions. An embedding is a dense vector that represents an item, a user, or a token, as in Chapters 10 and 11. A model is autoregressive when it generates its output one token at a time, with each new token conditioned on the input and on the tokens it has already produced. Distillation is the practice of training a smaller student model to reproduce the outputs of a larger teacher model, so that the student can be served at a cost the teacher could not meet.

Two families of models fall under the heading of generative recommendation. The first brings language models pre-trained on text to recommendation, to produce knowledge and features about items and users or to rank candidates through prompting and fine-tuning. The second trains recommendation models from scratch on behaviour sequences, borrowing the architectures, objectives, and scaling recipes of language models. HSTU, which models a user's actions as a sequence of tokens (Zhai et al., 2024), and TIGER, which retrieves items by generating their identifiers (Rajput et al., 2023), belong to the second family. Both families are generative in the same sense: they produce outputs, whether item codes, predicted actions, or text, as sequences of tokens, instead of only assigning a score to each item in a fixed list. The families are also converging. YouTube's PLUM adapts a pre-trained language model to generate item identifiers (He et al., 2025), and Meta's GR2 re-ranker reasons in text over items described by learned codes (Liang et al., 2026).

Generative models can enter every stage of the recommendation funnel of Chapter 2, from retrieval through ranking to re-ranking, and they can also work entirely offline, before any request arrives. Figure 12.1 maps these entry points to the sections of this chapter.

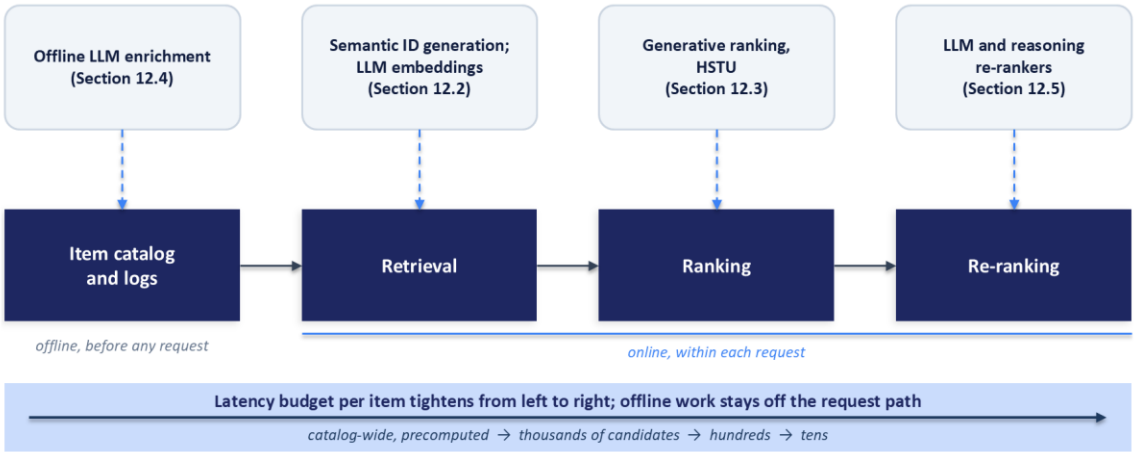


Figure 12.1: Where generative models enter the recommendation funnel. Offline enrichment (Section 12.4) prepares item and user knowledge before any request arrives. Semantic ID generation and LLM embeddings serve retrieval (Section 12.2), generative ranking models such as HSTU replace or join the ranker (Section 12.3), and language models and reasoning models re-rank short lists (Section 12.5). The latency budget per item tightens from left to right.

The figure also carries the frame that governs every design choice in this chapter: each stage has its own latency and cost budget. Offline enrichment runs in batch jobs, where a model can spend seconds on

each item without any user waiting. Retrieval must search millions of items within tens of milliseconds, ranking must score hundreds or thousands of candidates within a similar budget, and re-ranking orders a short list. A forward pass of a language model with billions of parameters over a few hundred tokens costs orders of magnitude more than the dot product that a two-tower retriever (Chapter 5) computes for each item, so placing such a model on the request path requires engineering to contain its cost.

A common belief in earlier years held that language models should never sit on the request path. Published systems now show that they can when the cost is contained. LinkedIn serves feed retrieval from language model embeddings computed in nearline pipelines (Danchev, 2026), YouTube serves retrieval candidates from a fine-tuned language model that generates item identifiers (He et al., 2025), and Instacart answers frequent search queries from a cache filled offline by a large model and sends the rare remainder to a small fine-tuned model in real time (Zhu et al., 2025). The levers are consistent across these systems: move work offline or nearline, cache results, distil large models into small ones, share computation across the candidates of one request, and limit expensive models to short lists. Chapter 14 treats latency engineering in general; this chapter shows how those levers apply to generative models.

Sections 12.2 to 12.5 follow the funnel of Figure 12.1, and Sections 12.6 to 12.8 cover evaluation, the choice among patterns, and risks and operations. The chapter's companion code in the book's repository implements small versions of several of these components, and the exercises in Section 12.10 build on it.

12.2 Semantic IDs and Generative Retrieval

A Semantic ID is a short tuple of integers, called codes, that identifies an item and is learned from the item's content, so that similar items share codes. This section builds Semantic IDs step by step, shows how a sequence model retrieves items by generating their Semantic IDs, and then follows Semantic IDs into ranking models and into language models.

12.2.1 From Arbitrary IDs to Semantic Addresses

An arbitrary item identifier is like a cloakroom ticket: it tells the system which coat to fetch, and nothing about the coat. A Semantic ID behaves more like a postal address, read from coarse to fine. The first code places the item in a broad region of the catalog, the second narrows the region, and later codes separate individual items. Two items whose Semantic IDs begin with the same codes are close in the content space from which the codes were learned.

Rajput et al. (2023), who introduced Semantic IDs for generative retrieval in the TIGER model, illustrate the hierarchy on a beauty-products dataset. With a small first-level codebook, most items whose first code was 3 were hair products, and most items whose first code was 1 were makeup and skin products for the face, lips, and eyes. That alignment with human categories is a tendency of the learned codes. The codes are learned without any taxonomy, so a first-level region can mix categories that the content encoder considers similar, and one human category can spread over several regions.

Semantic IDs also save memory. A tuple of L codes, each drawn from a codebook of K entries, can address K^L distinct items while a model stores only $L \times K$ code embeddings. TIGER used four codes with 256 entries each, so its vocabulary held 1,024 item tokens, enough to address $256^4 = 2^{32}$, or about 4.3 billion, identifiers (Rajput et al., 2023). An embedding table with one row per item grows with the catalog; a table of code embeddings does not.

12.2.2 Residual Quantization, One Level at a Time

Semantic IDs are usually produced by residual quantization, which applies vector quantization repeatedly. Vector quantization replaces a vector with the nearest entry, called a codeword, from a fixed set of vectors called a codebook, and records the index of that entry. One round of vector quantization with a few hundred codewords can only separate a few hundred groups. Residual quantization therefore quantizes what the first round missed. It subtracts the chosen codeword from the vector, which leaves a residual, and quantizes the residual with a second codebook, then repeats for as many levels as needed.

Starting from the item's latent vector z, level d of residual quantization chooses a code c_d and passes on a new residual r_d:

$$c_d = \arg\min_k \|r_{d-1} - e_k^{(d)}\|_2, r_d = r_{d-1} - e_{c_d}^{(d)}, r_0 = z$$

Here $e_k^{(d)}$ is the k-th codeword of the level-d codebook. The Semantic ID is the tuple $(c_1, c_2, \ldots, c_L)$, and the sum of the chosen codewords approximates the original vector:

$$\hat{z} = \sum_{d=1}^{L} e_{c_d}^{(d)}$$

Figure 12.2 works through two levels in two dimensions. The first level assigns the item to its nearest centroid, labelled c2 in the figure, so $c_1 = 2$, and the second level quantizes the leftover offset with a separate codebook whose codewords are much closer together, because residuals are much smaller than the original vectors. That shrinking scale is why each level needs its own codebook.

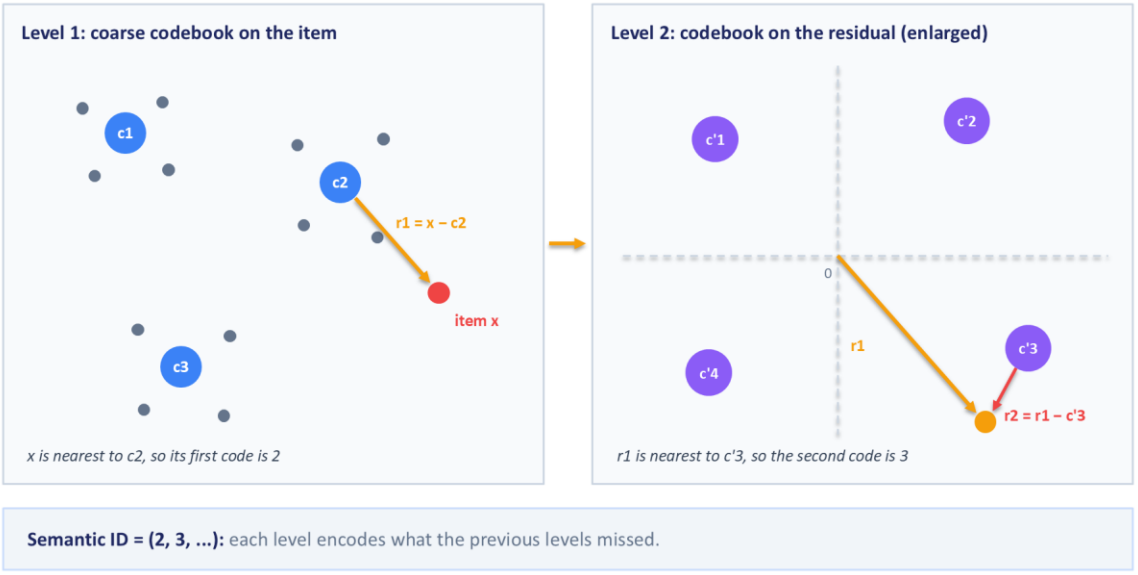


Figure 12.2: Residual quantization worked in two dimensions. Left: the first-level codebook has three centroids, and item x is closest to c2, so its first code is 2 and the residual r1 = x - c2 is passed on. Right: a second, finer codebook quantizes r1, choosing c'3, which leaves r2 = r1 - c'3. The Semantic ID (2, 3, ...) records the choice at each level, and each level encodes what the previous levels missed.

Residual quantization can be run with plain k-means clustering at each level, fitting k-means to the vectors, then to the residuals, and so on. Kuaishou's OneRec uses this residual k-means approach

(OneRec Team, 2025), and so does Spotify's podcast retrieval system (D'Amico et al., 2026). The alternative that TIGER popularized learns an encoder and the codebooks jointly in a residual-quantized variational autoencoder.

12.2.3 Learning the Codebooks with an RQ-VAE

A residual-quantized variational autoencoder (RQ-VAE) was first proposed for image generation (Lee et al., 2022), and TIGER adapted it to item embeddings (Rajput et al., 2023). The model has three parts, shown in Figure 12.3: an encoder network maps an item's content embedding x, such as a sentence-transformer embedding from Chapter 10, to a smaller latent vector z; a residual quantizer turns z into codes and into the quantized vector $\hat{z}$; and a decoder network maps $\hat{z}$ back to a reconstruction $\hat{x}$. TIGER encoded each item's title, price, brand, and category with a pre-trained sentence encoder, compressed the 768-dimensional result to a 32-dimensional latent vector, and quantized it with three levels of 256 codewords each.

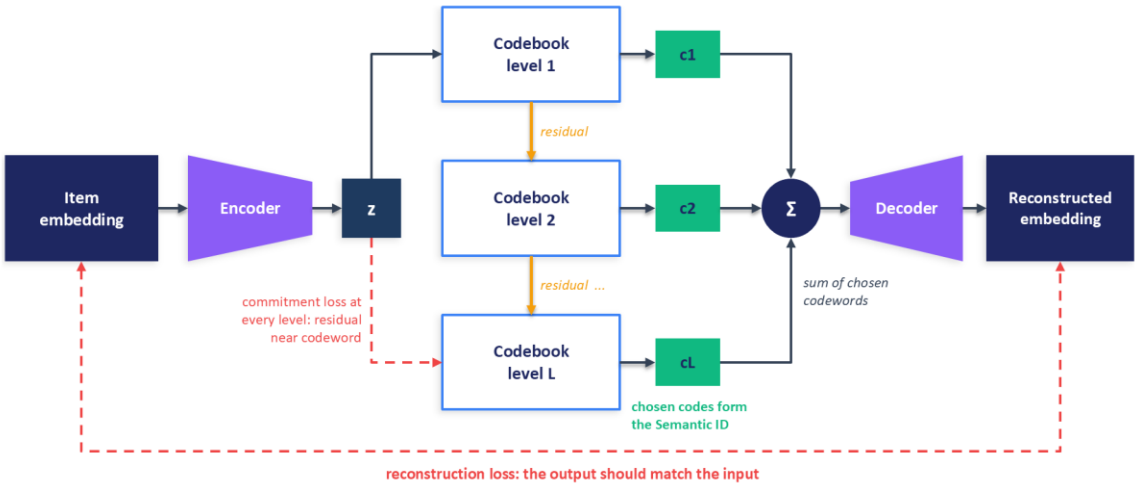


Figure 12.3: RQ-VAE architecture. An encoder compresses the item embedding to a latent vector z, three codebooks quantize z and its residuals in turn, and the sum of the chosen codewords is decoded back into a reconstruction of the input. The reconstruction loss trains the encoder and decoder, and the commitment loss keeps each residual close to its chosen codeword. The chosen codes (c1, c2, ..., cL) form the item's Semantic ID.

The training loss combines reconstruction with a quantization term at each level. TIGER uses

$$\mathcal{L} = \|x - \hat{x}\|^2 + \sum_{d=1}^{L} \left(\|\text{sg}[r_{d-1}] - e_{c_d}^{(d)}\|^2 + \beta \|r_{d-1} - \text{sg}[e_{c_d}^{(d)}]\|^2 \right)$$

where sg[·] is the stop-gradient operator, which passes its argument through unchanged in the forward pass and blocks gradients in the backward pass. The first term inside the sum moves each chosen codeword toward the residual it quantized. The second term, the commitment loss, moves the encoder's output toward the chosen codewords so that the encoder does not drift away from them, and β (0.25 in TIGER) sets its weight.

Choosing the nearest codeword is an argmin, which has no useful gradient, so the reconstruction loss cannot reach the encoder through the quantizer in the ordinary way. The straight-through estimator, used in VQ-VAE (Van den Oord et al., 2017), solves this by using the quantized vector in the forward pass and copying the gradient straight to the unquantized vector in the backward pass. In code, the decoder receives $z + \text{sg}[\hat{z} - z]$, which equals $\hat{z}$ numerically and has the gradient of z.

Codebooks can also be trained without gradients. Instead of the codebook term in the loss, each codeword is updated as an exponential moving average (EMA) of the vectors assigned to it in recent batches, which is how SoundStream trains the codebooks of its residual quantizer (Zeghidour et al., 2021). The chapter's companion code takes this route. Its codewords are buffers updated by moving averages, so each quantizer returns its chosen codewords as constants, and the encoding loop runs the residual quantization of Section 12.2.2:

```
z_e = self.encoder(x)
residual = z_e
z_q = torch.zeros_like(z_e)
codes: List[torch.Tensor] = []
commitment = z_e.new_zeros(())
for quantizer in self.quantizers:
    codeword, level_codes, level_loss = quantizer(residual)
    z_q = z_q + codeword
    residual = residual - codeword
    codes.append(level_codes)
    commitment = commitment + level_loss
```

The forward pass then decodes the straight-through form described above:

```
z_st = z_e + (z_q - z_e).detach()
x_hat = self.decoder(z_st)
```

Because every codeword is a constant, each residual is the encoder output minus constants, with the identity as its gradient, so every level's commitment loss trains the encoder, and the reconstruction loss reaches the encoder once, through the straight-through term. Two tempting variants break this without raising an error. If the quantizer returned its output in straight-through form and the loop subtracted it as is, the new residual for a residual r and codeword e would be $r - (r + \mathrm{sg}[e - r])$, whose gradient is zero, so levels after the first would stop training the encoder. Detaching the whole residual, as in `residual = (residual - codeword).detach()`, cuts later levels off by a different route. In both cases the model trains, and only the codebook statistics in the next subsection reveal the damage.

12.2.4 Dead Codes and Codebook Collapse

A dead code is a codeword that no input selects. Codebook collapse is the extreme case, in which most inputs map to a handful of codewords and the rest sit unused. Collapse matters for recommendation in a specific way. If only a few first-level codes are alive, the first code splits the catalog into a few huge groups, the later codes must do all the separating, and many items end up sharing complete Semantic IDs.

Collapse usually begins at initialization. Codewords initialized at random may lie far from every input, and a codeword that never wins an assignment never receives an update, under either the loss-based or the EMA training rule. The standard remedy is to initialize each codebook with k-means on the first training batch, so that every codeword starts near real data. TIGER does this to prevent collapse, following SoundStream (Rajput et al., 2023; Zeghidour et al., 2021). SoundStream adds a second remedy that continues during training: when the moving average of a codeword's assignment count falls below a small threshold, the codeword is replaced by an input vector sampled at random from the current batch (Zeghidour et al., 2021).

Some EMA implementations add a small constant to every assignment count before dividing, a step often called Laplace smoothing. It prevents division by zero for unused codewords, but it does not move them toward the data, so it does not revive dead codes. The companion code's quantizer uses this smoothing and both remedies, k-means initialization on a sample of training items and a reset of codewords whose moving-average usage falls well below the average, and its training script can switch each remedy off, which makes it a useful object for the codebook-health exercise in Section 12.10.

Collapse is easy to measure. For each level, report the fraction of codewords used by at least one item, the entropy of the code distribution (how evenly items spread over codes), the size of the largest group under each first-level code, and the number of items that share a full Semantic ID with another item. Rajput et al. (2023) trained their RQ-VAE until codebook usage reached at least 80 percent. Wang et al. (2024b) identify a code assignment bias, in which a few codes receive a disproportionate share of items, and their LETTER tokenizer adds a diversity loss and an alignment with collaborative signals to counter it. When Kuaishou compared tokenizers for OneRec, residual k-means used its codebooks more fully and spread items more evenly across codes than an RQ-VAE did (OneRec Team, 2025).

12.2.5 Prefix Trees, Collisions, and Invalid Sequences

Because Semantic IDs are read from coarse to fine, the set of all item IDs in a catalog forms a prefix tree, or trie: each level of the tree corresponds to one code position, each path from the root to a leaf spells out one Semantic ID, and items that share a prefix share a branch. Figure 12.4 draws a small catalog as a trie and marks three situations that every Semantic ID system must handle.

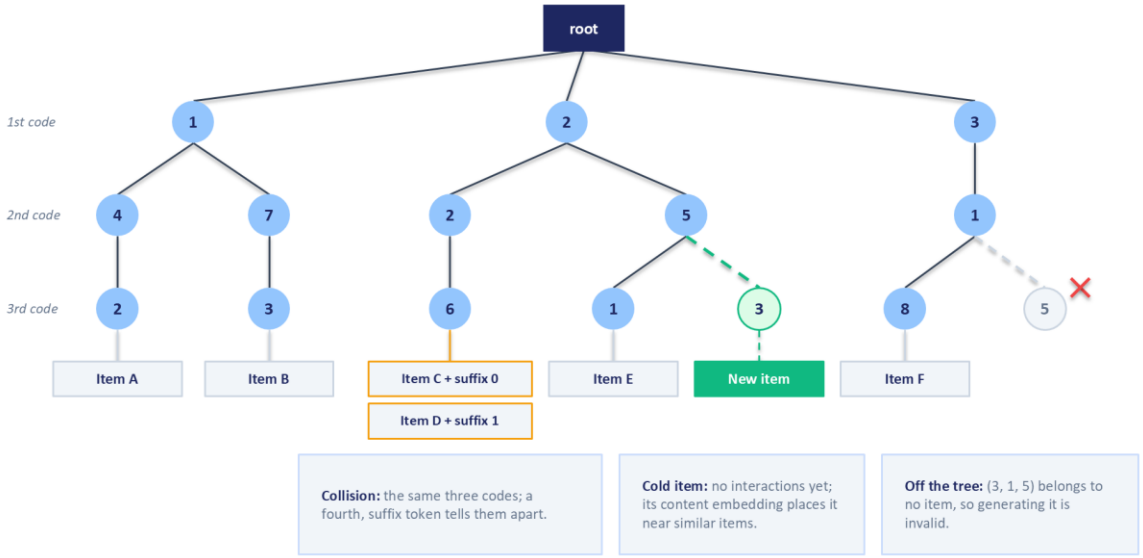


Figure 12.4: A catalog's Semantic IDs as a prefix tree. Each level of the tree corresponds to one code position, and items sit at the leaves. Two items that share all three codes collide and are separated by a fourth, suffix token. A new item attaches under the existing branch that matches its content. A generated sequence whose path leaves the tree is invalid, because no item carries that Semantic ID.

A collision occurs when two or more items receive the same Semantic ID. Collisions are a property of the codebook and the catalog, fixed once after the quantizer is trained. TIGER resolves them by appending a final token that numbers the colliding items, so two items that share $(12, 24, 52)$ become $(12, 24, 52, 0)$ and $(12, 24, 52, 1)$, with a lookup table from Semantic IDs to items (Rajput et al., 2023). A healthy codebook keeps collisions rare, so the extra token takes small values.

An invalid sequence is a different problem. It arises at generation time, when a model that emits codes one at a time produces a tuple that belongs to no item. The space of possible tuples is vast compared with any catalog: TIGER's four codes of 256 entries span more than four billion tuples for datasets of 10,000 to 20,000 items. Rajput et al. (2023) observed that their model almost always produced valid IDs, with invalid IDs making up roughly 0.1 to 1.6 percent of top-10 retrievals across their three datasets, and they handled the remainder by widening the search and filtering. A production system cannot assume a small rate, because every invalid sequence wastes a slot in the candidate list. The reliable fix constrains generation to the trie, as the next subsection describes.

12.2.6 Generative Retrieval with TIGER

TIGER, the Transformer Index for GEnerative Recommenders, treats retrieval as sequence-to-sequence generation (Rajput et al., 2023). Each item in a user's history is replaced by its Semantic ID, the codes are concatenated into one token sequence, and an encoder-decoder transformer is trained to generate the Semantic ID of the next item, one code at a time. TIGER also prepends a hashed user token to personalize the output. Training uses ordinary next-token cross-entropy, exactly as in a language model, with the vocabulary of code tokens in place of words. Rajput et al. (2023) describe the result as retrieval without a separate index: the model's parameters act as the index, since generating a Semantic ID is the same act as locating an item.

At inference, the decoder must return a list of items. Beam search does this by keeping the B most probable partial sequences at each step, extending each with every possible next code, and keeping the B best extensions by cumulative log probability. After L steps the beam holds B complete Semantic IDs, ranked by probability. Constrained beam search restricts the extensions at each step to codes that keep the partial sequence inside the catalog's trie, by setting the probability of every other code to zero. GENRE introduced this technique for retrieving entities by generating their names (De Cao et al., 2021), and it applies unchanged to Semantic IDs. Figure 12.5 shows the full pipeline.

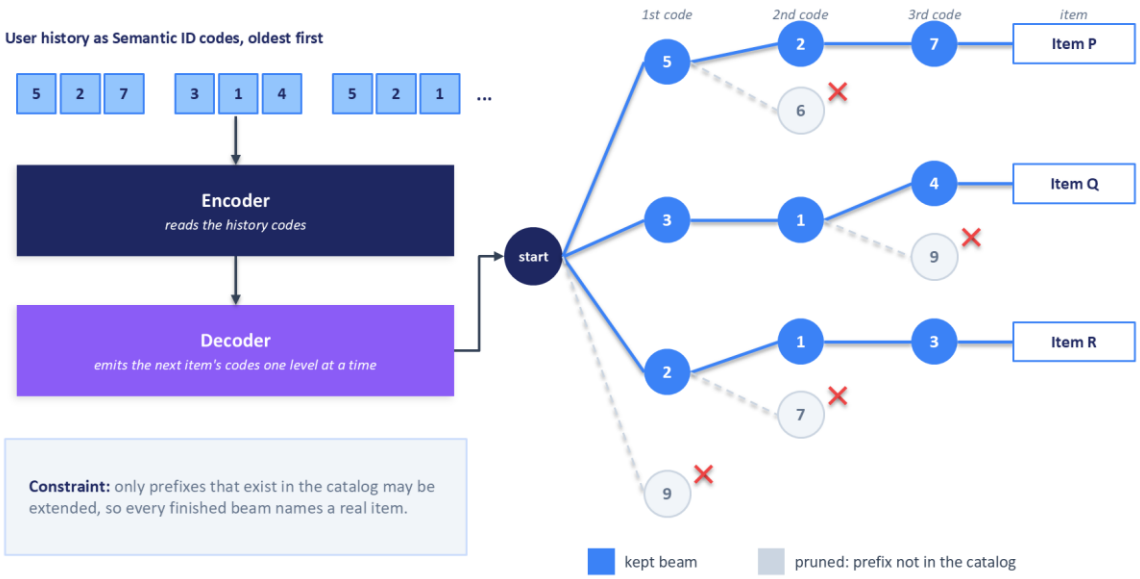


Figure 12.5: Generative retrieval with constrained beam search. The encoder reads the user's history as a sequence of Semantic ID codes. The decoder generates the next item's codes one level at a time, and at each step the beam keeps only extensions that remain inside the catalog's prefix tree, so branches that would leave the tree are pruned and every finished sequence names a real item.

Constrained decoding guarantees validity and costs a trie lookup per step. It does not make generative retrieval cheaper than embedding retrieval. A two-tower retriever (Chapter 5) answers a request with one encoder pass and an approximate nearest-neighbour search, while beam search runs the decoder L times for each of B beams, and Rajput et al. (2023) note that inference can therefore cost more than nearest-neighbour search. What generative retrieval offers in exchange is a model whose memory does not grow with the catalog, a shared representation for similar items, and a decoder that conditions every generated code on the whole history through cross-attention, which a dot product between two separately computed vectors cannot do.

12.2.7 What Semantic IDs Do and Do Not Do for Cold Start

Semantic IDs are often presented as a cure for item cold start, and the truth is more limited. A new item does receive a meaningful Semantic ID the moment its content is embedded and quantized, with no interactions required, even when the encoder was fine-tuned on co-engagement data as in Chapter 10. The new item lands in a branch of the trie next to similar items, as Figure 12.4 shows.

Whether a generative retriever will ever generate that new item's full Semantic ID is another matter. The decoder learned to produce the IDs that appeared as targets during training, and its probabilities for the final codes of an unseen item are not trained. TIGER's cold-start experiment works around this: when the model generates a Semantic ID, unseen items that share its first three codes are added to the retrieved list, with a hyperparameter capping the share of unseen items (Rajput et al., 2023). The prefix does the work, and the cap is a policy choice. In the LIGER study on several public benchmarks, Yang et al. (2024) found that TIGER-style models performed close to zero on cold-start items in most datasets, because the decoder's probabilities overfit to the items seen in training, while dense retrievers given the same text kept a useful level of cold-start performance. Their remedy is a hybrid that generates a small candidate set by beam search, adds the cold-start items, and ranks the combined set with dense representations. Semantic IDs give a new item an address, and a retrieval system still needs a deliberate path, such as prefix matching, a dense retriever, or an exploration slot, to deliver the item to users.

12.2.8 Semantic IDs as Ranking Features

Semantic IDs need not be generated at all to be useful. Singh et al. (2024) used them as features in a production ranking model for YouTube, in place of the randomly hashed video identifier. The difficulty is that a Semantic ID is compact, and a ranking model memorizes through large embedding tables. Their solution splits each Semantic ID into pieces, hashes each piece into an embedding table, and sums the resulting embeddings. Pieces can be fixed n-grams of codes, such as each single code and each pair of adjacent codes, or they can be learned with SentencePiece, the subword tokenizer used by many language models, so that frequent code combinations get their own embeddings. In their setup, with eight levels of 2,048 codes each, SentencePiece pieces outperformed hand-chosen n-grams, and Semantic IDs replaced the video identifier while improving generalization to new and long-tail videos without lowering overall quality.

This is often the cheapest way to benefit from Semantic IDs, because it changes a feature in an existing ranker and leaves retrieval, serving, and the model architecture alone. Its effect should be measured on new and long-tail items separately, since the gains Singh et al. (2024) describe concentrate in those slices.

12.2.9 Language Models in Retrieval

Language models enter retrieval in two further ways. The first uses a language model as the encoder of a two-tower retriever. LinkedIn fine-tuned a language model as a dual encoder that turns text prompts describing members and posts into embeddings, trained with a contrastive loss that pairs each engaged post with randomly sampled posts and with posts that the member saw but ignored (Danchev, 2026). LinkedIn reports that engagement counts written into prompts as digits carried almost no signal into the embeddings until each count was converted to a percentile bucket wrapped in dedicated marker tokens, a reminder that language models read numbers as text. Embeddings are refreshed nearline and searched with an exact nearest-neighbour search on GPUs, so no language model runs at request time.

The second way teaches a language model to generate Semantic IDs. YouTube's PLUM extends a pre-trained language model's vocabulary with Semantic ID tokens, continues pre-training on sequences that mix those tokens with text, and fine-tunes the model to generate the Semantic IDs of videos a user will watch (He et al., 2025). PLUM runs in production as a retrieval source for long-form videos and Shorts. Spotify's GLIDE builds Semantic IDs from podcast episode text alone, injects a collaborative-filtering user embedding into a compact open-weight language model as a soft prompt, which is a learned vector placed among the input token embeddings, and fine-tunes the model with low-rank adapters (Section 12.5) to generate episodes for discovery (D'Amico et al., 2026). Because recommendations are requested on user events, Spotify can generate them ahead of time and cache them. Both systems treat the language model as one more retrieval source, evaluated by adding its candidates to the existing pool in live experiments, which is the integration pattern that Section 12.7 recommends for a first deployment.

12.3 Generative Ranking and Recommendation Foundation Models

Section 12.2 generated item identifiers for retrieval. This section turns to ranking, where the most influential generative design comes from Meta: Hierarchical Sequential Transduction Units, or HSTU, the architecture behind Meta's Generative Recommenders (Zhai et al., 2024). The section then surveys the recommendation foundation models that Netflix, Kuaishou, LinkedIn, and Meta's advertising teams have described, and closes with what a smaller team can take from them.

12.3.1 Actions as Tokens

A conventional deep learning recommendation model, or DLRM, ranks a candidate item by combining thousands of features about the user, the item, and their history: identifiers, categorical attributes, and numerical features such as decayed counters and click-through ratios. Zhai et al. (2024) reformulate ranking as sequence modelling over a single time series per user. Categorical features that change slowly are merged into the main sequence of interactions, and numerical features are dropped. Their argument for dropping numerical features is that a counter such as a user's click-through rate on a topic is an aggregate over events that the sequence already contains, so a sufficiently expressive model reading a long enough history can learn the same signal.

The sequence interleaves items and actions. Writing Φ_i for the i-th item shown to the user and a_i for the user's action on it, such as a click, a like, a skip, or a completed view, the history reads $\Phi_0, a_0, \Phi_1, a_1, \ldots$ in time order. Ranking asks what the user will do with a candidate item Φ_{i+1}, so the candidate is appended to the history and the model predicts the next action token:

$$p(a_{i+1} \mid \Phi_0, a_0, \Phi_1, a_1, \ldots, \Phi_{i+1})$$

This layout makes the ranker target-aware: the candidate interacts with every event in the history through attention, inside the encoder, instead of meeting a precomputed user vector only at the end, as it would in a two-tower model. A small network on top of the output at the candidate's position produces the multi-task predictions, such as click and like probabilities, that Chapter 7 combines into a ranking score. Retrieval uses the same backbone differently: the model predicts the next item a user will engage with positively, trained only on positions where the action was positive.

12.3.2 Inside an HSTU Layer

Chapter 11 explained the attention layers of SASRec (Kang and McAuley, 2018), in which queries, keys, and values come from linear projections and a softmax turns query-key scores into weights that sum to one. HSTU keeps the stacked, residual structure of a transformer and changes three things. Each layer first projects its input X with a single linear layer f_1, applies the SiLU activation function, $\mathrm{SiLU}(x) = x \cdot \sigma(x)$, and splits the result four ways:

$$U(X), V(X), Q(X), K(X) = \mathrm{Split}\big(\phi_1(f_1(X))\big)$$

It then aggregates over the sequence with pointwise attention:

$$A(X)V(X) = \phi_2(Q(X)K(X)^\top + \mathrm{rab}^{p,t})V(X)$$

and finally gates and projects the result:

$$Y(X) = f_2(\mathrm{Norm}(A(X)V(X)) \odot U(X))$$

Both ϕ_1 and ϕ_2 are SiLU, Norm is layer normalization, and $\odot$ is elementwise multiplication.

The first change is the absence of softmax. Each attention weight is computed independently, so the weights need not sum to one. Zhai et al. (2024) motivate this with the intensity of preference: how many past events relate to a candidate is itself a strong signal, and softmax normalization erases it by rescaling the weights. They also argue that softmax suits a fixed vocabulary better than the constantly changing item vocabulary of a recommender. The second change is the relative attention bias $\mathrm{rab}^{p,t}$, a learned term added to each query-key score that depends on the distance between two events in position and in time, typically through buckets of the time gap. Two events a minute apart and two events a month apart can then interact differently even when they sit next to each other in the sequence. The third change is the gate $U(X)$, which multiplies the attention output elementwise and plays the role that explicit feature-interaction layers play in a DLRM.

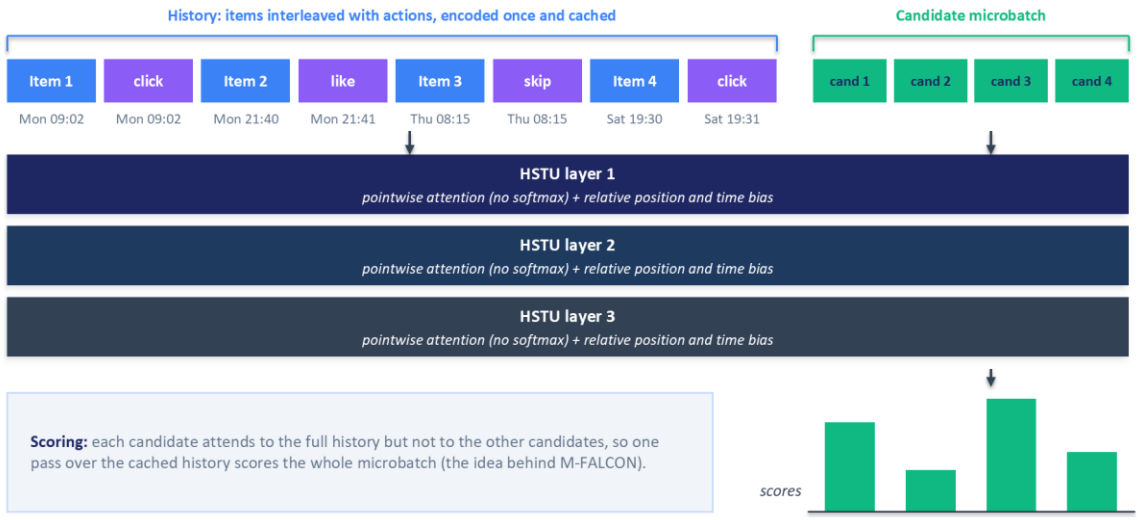


Figure 12.6: HSTU applied to target-aware ranking. The user's history interleaves item tokens with action tokens such as click, like, and skip, each carrying a timestamp. Stacked HSTU layers apply pointwise attention with a relative bias for position and time. Candidate items are appended after the history, and because the history is encoded once and reused, a microbatch of candidates is scored together, which is the idea behind M-FALCON.

The chapter's companion code in the book's repository includes a compact HSTU-style encoder, which fuses each item with its action into one token and keeps pointwise SiLU attention and a time-bucketed relative bias, and which can be trained on a single machine. Exercise 3 in Section 12.10 uses it to study how candidates can share one encoded history.

12.3.3 Scoring Thousands of Candidates

Target-aware ranking has an obvious cost. If every candidate must be appended to a history of n events and run through the encoder, scoring m candidates costs m full passes. Meta's answer is M-FALCON, short for Microbatched-Fast Attention Leveraging Cacheable OperatioNs (Zhai et al., 2024). Within one forward pass, M-FALCON appends a microbatch of b_m candidates to the history and modifies the attention mask and the relative bias so that each candidate attends to the history and to itself, never to the other candidates. Each candidate then receives exactly the score it would have received alone, and the cost of the pass grows as $(n + b_m)^2$ instead of $b_m n^2$. The remaining candidates are processed in further microbatches, and the history's keys and values are stored in a key-value cache, so later microbatches reuse them instead of recomputing them. A key-value (KV) cache stores the attention keys and values computed for earlier tokens; language models use one to avoid recomputing the prompt when generating each new token. Zhai et al. (2024) report that HSTU's efficiency and M-FALCON together let them serve a ranking model 285 times more computationally complex than the DLRM it replaced while handling more queries per second on the same inference budget.

LinkedIn's feed ranker relies on the same idea. Its Generative Recommender ranks posts by reading more than a thousand of a member's past interactions, and its serving stack computes the history representation once and scores all candidates in parallel with custom attention masks, a technique LinkedIn calls shared context batching (Danchev, 2026).

12.3.4 Scale and Deployment at Meta

Zhai et al. (2024) report that HSTU-based Generative Recommenders with 1.5 trillion parameters improved online metrics by 12.4 percent in A/B tests and were deployed on multiple surfaces of a large internet platform with billions of users. They also found that model quality followed a power law in training compute across three orders of magnitude, while the DLRMs they compared against plateaued. One of their baselines is instructive for this chapter: a generative recommender restricted to content features performed far worse than models given the history of user actions, which is the point of the paper's title, "Actions Speak Louder than Words".

Meta has continued the line. ULTRA-HSTU applies 18 layers of self-attention to histories of 16,000 events, using a sparse attention pattern whose cost grows linearly with sequence length, and is deployed in a production environment serving billions of users (Ding et al., 2026). Its authors report large gains in training and inference scaling efficiency, the slope of model quality against compute, over the original HSTU, and attribute 4 to 8 percent gains in consumption and engagement to the deployment. These numbers come from Meta's systems and Meta's data, and they show that the approach can scale on Meta's platform.

12.3.5 Recommendation Foundation Models Beyond Meta

A recommendation foundation model is a large model trained on a platform's complete interaction history and then reused by many downstream models, the way a pre-trained language model is reused for many tasks. Netflix describes a foundation model trained autoregressively on member interaction sequences (Hsiao et al., 2025). An interaction tokenization step first merges adjacent raw events into higher-level tokens, in analogy with byte-pair encoding in language models. The model predicts the next item and, as auxiliary targets, attributes such as genre. For new titles, Netflix mixes a learned embedding of each title's identifier with an embedding computed from its metadata, weighting the metadata more heavily for young titles. Downstream teams use the model directly, as a source of embedding features, or as a base for fine-tuning, and Netflix keeps the embedding space stable across retraining so that each new version does not break its consumers.

Meta's advertising organization pushed the idea toward knowledge transfer. Its Generative Ads Recommendation Model, GEM, is trained at a scale comparable to large language models and passes what it learns, through distillation and related post-training techniques, to the many smaller ads ranking models that serve traffic. Meta reports increases in ad conversions on Instagram and Facebook from the deployment (Li et al., 2025). The pattern, a very large model that trains while smaller models serve, recurs throughout this chapter.

Kuaishou's OneRec goes furthest in replacing the funnel (OneRec Team, 2025). An encoder reads a user's long behaviour history, a decoder with mixture-of-experts layers, which route each token to a few of many expert sub-networks, generates the Semantic IDs of recommended videos directly, and reinforcement learning against a reward model aligns the generated lists with user preferences. OneRec handles about a quarter of the requests in the main feeds of the Kuaishou and Kuaishou Lite apps. Kuaishou reports that the generative model alone matched the multi-stage system it replaced on that traffic, at about a tenth of its operating cost. OneRec is the clearest published example of end-to-end generative recommendation in production, and also of the infrastructure, from custom inference kernels to a dedicated reinforcement learning service, that such a system requires.

12.3.6 What Smaller Teams Can Take From This

Every system in this section rests on years of logged interactions, fleets of GPUs for training and serving, and teams that maintain streaming training pipelines. Few organizations have all three, and none of the published results transfers automatically to a different platform. Three lessons transfer anyway. First, log interactions as sequences with timestamps and action types, including negative actions such as skips, because every generative ranker consumes exactly that format and it cannot be reconstructed later. Second, sequential models of moderate size, such as the SASRec-style encoders of Chapter 11 or a compact HSTU-style encoder, give a team the experience of training and serving sequence models before it commits to larger ones. Third, the shared-history scoring idea applies to any target-aware sequence ranker, at any scale, and it is often the difference between an affordable model and an unaffordable one. Section 12.7 returns to the question of when a team should adopt generative ranking.

12.4 LLMs as Offline Feature and Label Generators

The safest place for a large language model in a recommender is outside the request path. Run offline, a language model can spend seconds on each item and write structured knowledge that a conventional ranker consumes as ordinary features; no user waits for it, and the cost is paid once per item or per user instead of once per request. This section covers what to generate, how to turn generated text into features without corrupting them, and how teams keep the cost of generation under control.

12.4.1 Knowledge the Logs Do Not Contain

The KAR framework of Xi et al. (2024) is a clean template for offline enrichment. KAR asks a language model for two kinds of knowledge: factual knowledge about each item, and reasoning knowledge about each user's preferences, inferred from the user's history. Asking a model directly why a user likes what they like produces vague answers, so KAR uses factorization prompting, which breaks the question into factors that matter in the domain, such as genre, actors, directors, theme, and mood for films, and asks about each one. The generated text is then encoded into vectors by a knowledge encoder, and a hybrid-expert adaptor, a small network that combines several expert subnetworks, maps those vectors from the semantic space of the language model into the space the recommender uses. The resulting augmented vectors are added as features to an existing ranking model, whatever its architecture. Everything is generated and stored in advance, so serving costs only a lookup. Xi et al. (2024) report deploying KAR on Huawei's news and music recommendation platforms, with online gains in both.

Two design choices in KAR are worth copying even without the rest of the framework: the adaptor, because text embeddings and recommendation features live in different spaces and something must learn the mapping between them, and the precomputation, because a design that needs generation at request time inherits its slowness. Both return in the rest of this section.

12.4.2 Designing Item Knowledge Prompts

Item knowledge prompts should ground the model in the item's own metadata, so that it describes the product in front of it instead of a similar product it remembers, and they should ask for output short and regular enough for a downstream model to use. The chapter's companion code asks for a two-sentence description, filled in once per item. Its prompt function shortens the title to 160 characters and the description to 300, and returns:

```
return (
    "Describe this product for a shopping recommender in two sentences: its category,
"
    "what it is used for, and the kind of shopper who buys it. Use only the details
given.\n"
    f"Title: {short_title(title, 160)}\n"
    f"Brand: {brand or 'unknown'}\n"
    f"Description: {desc or 'none'}"
)
```

Each part of the request serves a purpose. The category normalizes noisy merchant text into consistent terms, which helps items whose descriptions are missing or written in marketing language. The use and the likely shopper add a guess at intent that no catalog attribute records. The final instruction, together with explicit placeholders for a missing brand or description, discourages the model from filling gaps from memory. The companion code embeds each answer whole as one vector, so free text is enough. A pipeline that splits the output into separate fields, for example to feed complementary-item retrieval with generated categories, should ask for named fields in a fixed format and log and retry any response that ignores it.

Operationally, treat generated knowledge like any other derived dataset. Store each output with the prompt version, the model identifier, and a hash of the input text, so that a change to any of them triggers regeneration of exactly the affected items. Sample outputs for human review before the features reach a model, because a language model can state plausible facts about a product that are wrong. Chapter 4's discipline about point-in-time correctness also applies: a feature generated from a user's session must use only events that happened before the moment being predicted, or the model will learn from the future.

12.4.3 From Text to Features: Matching the Embedding Space

Generated text reaches a ranker as vectors, and the choice of encoder decides whether those vectors mean anything to the model. An embedding is meaningful only relative to the encoder that produced it. Chapter 10 fine-tuned a sentence encoder, such as the Sentence-BERT models of Reimers and Gurevych (2019), on co-engagement data, and the fine-tuned encoder places items in a different space from the base model it started from, even though both produce vectors of the same length. A cosine similarity between a vector from one encoder and a vector from the other is a number with no meaning. Any feature that compares two vectors, such as the similarity between an encoded session intent and an encoded candidate item, needs both from the same encoder. When generated text must sit alongside existing embeddings from a different encoder, a learned mapping such as KAR's adaptor, trained on the recommendation objective, is the honest way to connect them.

Three further failure modes are common and silent. The first is partial coverage. Generation is expensive, so a team may enrich only the most popular items, and the remaining items need an explicit missing value and a missing-indicator feature. A zero vector looks like real data to the model, which then learns that popular items have informative features and everything else has a peculiar constant one. The coverage of every generated feature, overall and by item segment, belongs on the same dashboard as its values. The second is misaligned joins. Generated features must be joined to training examples by stable keys, such as item and session identifiers, never by position in a list, because positional joins survive any reordering of the inputs without raising an error and then attach each feature to the wrong example. The third is misleading names. A feature should be named for how it was computed, so that a column described as a sequence-model user embedding actually comes from a sequence model. Each of

these bugs lets a model train and evaluate normally while learning something other than what its designers believe.

12.4.4 Cascades and Distillation

Running a large language model over a whole catalog, and again whenever the catalog changes, is often too expensive. A cascade keeps the cost down by sending each item first to the cheapest method that might handle it, such as rules or string matching, then to a small model, and only the hard remainder to a large language model. Distillation uses the large model's outputs as training labels for a small model that takes over most of the work. The two patterns reinforce each other: the large model labels the hard tail, those labels train the small model, and the small model's growing competence shrinks the tail. Figure 12.7 shows the resulting pipeline.

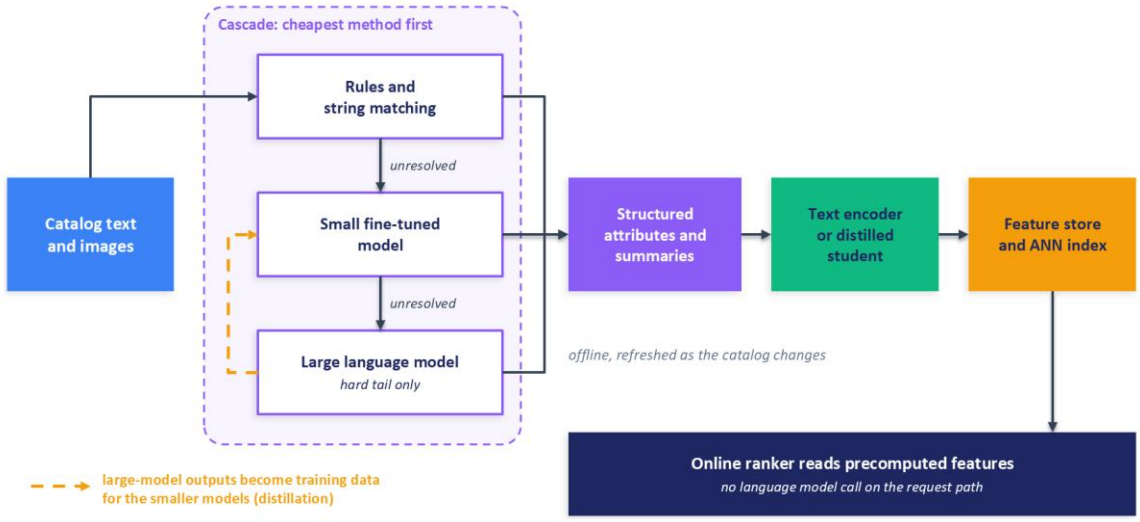


Figure 12.7: An offline enrichment pipeline built as a cascade. Catalog text and images enter a cascade that resolves easy cases with rules, passes harder cases to a small fine-tuned model, and sends only the hard tail to a large language model. The cascade's outputs are encoded and written to the feature store and nearest-neighbour index. The online ranker reads these precomputed features without calling a language model, and the large model's outputs become training data for the smaller models.

DoorDash's retail catalog work follows this shape closely (Xu and Vadrevu, 2024). To extract brands from merchant product data, an in-house classifier tags what it can, a language model handles the products it cannot tag confidently, and the confirmed brands become new training data for the classifier. To label organic products, a waterfall tries exact string matching first, then language model reasoning, and finally an agent that searches the web. To bootstrap a general attribute-extraction model, DoorDash passes the most similar human-labelled products to a large language model as examples for labelling new ones, and fine-tunes a language model on the result for scalable inference. The extracted attributes then serve as features in DoorDash's personalized ranking models.

Pinterest applied the same recipe to search relevance (Wang et al., 2024a). A language model fine-tuned as a cross-encoder, a model that reads the query and the document together, scores how relevant a Pin is to a query, using Pin text enriched with image captions generated by a vision-language model. This teacher is too slow for real-time serving, so it labels a large volume of logged search traffic, including languages that human annotators never labelled, and a feed-forward student learns from those labels and serves online. The recipe transfers directly to recommendation surfaces.

Instacart has published two variants. For search query understanding, an offline pipeline built on a large model fills a cache for the most frequent queries, and its outputs train a much smaller language model, fine-tuned with low-rank adapters, that handles the long tail of queries in real time (Zhu et al., 2025). For generated discovery content, a fine-tuned DeBERTa classifier, a compact encoder model, judged the relevance of products to generated themes at more than 99 percent lower cost than a closed-weight language model, which made it affordable to check every generated placement (Hasani et al., 2026). In both cases the large model sets the quality bar, and a small model carries the volume.

12.5 LLMs and Reasoning Models as Rankers and Re-rankers

Ranking a short list of candidates is the stage where a language model's knowledge can be applied most directly, because the candidates are few and their descriptions fit in a prompt. This section moves from prompting a general model, through fine-tuning, to the reasoning re-rankers that recent work trains with reinforcement learning, and it ends with the output checks and serving decisions that every one of these approaches needs.

12.5.1 Prompting a Language Model to Rank

A language model can be asked to rank in three ways, borrowed from learning to rank. A pointwise prompt presents one candidate and asks for a judgement about it, such as whether the user will like it. A pairwise prompt presents two candidates and asks which is better. A listwise prompt presents the whole candidate list and asks for an ordering. Comparing the three on four recommendation datasets, Dai et al. (2023) found ChatGPT stronger at listwise and pairwise ranking than at pointwise ranking, and recommended listwise prompts because one call ranks the whole list, which gives the best balance of cost and quality.

Hou et al. (2024) framed recommendation as conditional ranking: given a user's interaction history as the condition, order candidates supplied by a conventional retriever. Their study exposed several weaknesses of zero-shot language model rankers. The models struggled to use the order of the history, which prompts that emphasize the most recent interactions partly fixed. They were biased by the position of candidates in the prompt, which bootstrapping mitigated: the candidates are presented several times in shuffled orders and the resulting rankings are aggregated. They also favoured popular items, a bias that weakened when the prompt gave the model a shorter history, which it used more effectively. Finally, they occasionally returned items that were not among the candidates. Ranking quality depended strongly on which retriever produced the candidates and how hard they were to tell apart, a point Section 12.6 develops.

The chapter's companion code uses a listwise prompt, sent as a single user message in the model's chat template. Its prompt function numbers the session's history from oldest to newest, labels each candidate with a letter in square brackets, sets `last` to the letter of the final candidate, and returns:

```
return (
    "A shopper viewed these products in this order during one session:\n"
    f"{_history_block(history_titles)}\n\n"
    "Rank the candidate products by how likely the shopper is to view each one
next.\n"
    f"{candidates}\n\n"
    f"Answer with one line that starts with 'Ranking:' followed by all letters A to
{last}, "
```

```
    "most likely first, separated by commas. Do not explain."
)
```

Letters keep the output short and easy to parse, the `Ranking:` marker tells the parser where the answer starts, and the explicit instruction to rank every candidate reduces, without eliminating, responses that stop after a few items. Section 12.5.5 covers what to do when the model ignores the instruction anyway.

12.5.2 Fine-Tuning a Language Model as a Ranker

Zero-shot rankers inherit whatever notion of relevance their pre-training happens to contain. Bao et al. (2023) showed how weak that notion can be: in their movie and book recommendation tasks, general language models prompted with examples in context performed close to random guessing, with an area under the ROC curve near 0.5. Their TALLRec framework fine-tunes the model on recommendation data framed as a binary question: given a user's history, will the user like this item, answered with "Yes" or "No". After tuning, the model performed well with fewer than a hundred training examples and transferred from one domain to another, for example from movies to books.

Fine-tuning every weight of a model with billions of parameters is expensive, so TALLRec uses low-rank adaptation, or LoRA (Hu et al., 2022). LoRA freezes each pre-trained weight matrix W and learns a low-rank update to it:

$$h = Wx + \frac{\alpha}{r} BAx$$

where A has r rows, B has r columns, the rank r is small (often 8 to 64), and α is a scaling constant. Only A and B are trained. For a model with a few billion parameters, rank-16 adapters on two of the attention projections add a few million trainable parameters, roughly a tenth of a percent of the model, and the adapters can be merged into W after training so that serving costs nothing extra. QLoRA goes further by storing the frozen base model in 4-bit precision during training, which lets larger models be fine-tuned on a single GPU (Dettmers et al., 2023).

The companion code's TALLRec-style prompt asks the binary question directly, after the same numbered history:

```
return (
    "A shopper viewed these products in this order during one session:\n"
    f"{_history_block(history_titles)}\n\n"
    f"Candidate product: {short_title(candidate_title)}\n"
    "Will the shopper view the candidate product next? Answer Yes or No."
)
```

To rank a list, the companion code reads the model's next-token probabilities at the first answer position and scores each candidate by $P(\text{Yes})/(P(\text{Yes}) + P(\text{No}))$, which ignores probability mass on every other token. Scoring costs one forward pass per candidate, although the passes for one user can be batched.

12.5.3 Masking the Labels

When fine-tuning a causal language model on prompt and answer pairs, the loss should be computed only on the answer tokens. Otherwise most of the gradient goes into teaching the model to reproduce prompts, which are long and repetitive, and very little into the answer, which is one or two tokens. The

convention in PyTorch and Hugging Face libraries is to set the label of every position that should not contribute to the loss to -100, the cross-entropy loss's ignore index. With right padding, where padding tokens follow the text, the masking step looks like this:

```
# one example: input_ids has shape (seq_len,)
labels = input_ids.clone()
labels[attention_mask == 0] = -100   # padding positions
labels[:prompt_token_len] = -100     # prompt positions: train only on the answer and
end-of-sequence token
```

The subtle part is `prompt_token_len`. It must be the length, in tokens, of exactly the text that precedes the answer in the training sequence, including the chat template's markers for the start of the assistant's turn. The reliable way to compute it is to apply the same chat template to the prompt alone with the generation prompt appended, and tokenize that. If the length comes from a different string, such as the raw prompt without the template, the mask ends a few tokens early or late, and the model is trained either to predict template tokens or never to see the answer at all. With left padding, slicing from the start of the sequence masks the padding instead of the prompt, which is a second way to get the same bug. Neither mistake raises an error, and the training loss still falls. A unit test that decodes the unmasked tokens of a few training examples and checks that they read "Yes" or "No" followed by the end-of-sequence token catches both, and Exercise 5 in Section 12.10 asks you to write one.

12.5.4 Generative Reasoning Re-rankers

Reasoning models are language models trained to write out intermediate reasoning, a chain of thought, before giving a final answer. Applied to re-ranking, a reasoning model reads a user's history and a pre-ranked candidate list, explains which candidates fit, and emits a reordered list. Meta's GR2, the Generative Reasoning Re-ranker, trains such a model in three stages (Liang et al., 2026). A pre-trained language model is first mid-trained on Semantic IDs, built so that at least 99 percent of items receive unique IDs, so that the model can name catalog items in its own vocabulary. It is then fine-tuned on reasoning traces from a larger teacher, filtered by rejection sampling so that only traces whose prediction matches the item the user engaged with next are kept. Finally, it is trained with reinforcement learning against verifiable rewards, computed by a program that compares the produced ranking with the user's actual engagement.

The GR2 work is most useful to practitioners for what went wrong. The model learned to hack its reward by returning the candidates in the order it received them, which earned the reward for a well-formed output without any re-ranking, and the authors made the format reward conditional so that copying the input order no longer paid (Liang et al., 2026). In a later technical report on taking GR2 to industrial scale, Li et al. (2026) replaced supervised fine-tuning on teacher traces, which collapsed at that scale, with on-policy distillation, in which the student generates its own outputs and the teacher supplies a target for every token of them. They also trained a policy that ranks without writing out its reasoning, and report that it matched the reasoning model's quality at a fraction of its serving cost.

Reasoning is expensive at inference because an autoregressive decoder spends one sequential step on every reasoning token. Diffusion-GR2 converts the GR2 re-ranker into a block-diffusion language model, which decodes many positions in parallel and refines them over a few denoising steps, and then fine-tunes it to produce valid permutations, because positions decoded independently can repeat, drop, or invent candidates (Zhang et al., 2026). This line of work is recent and comes from one research group, so its techniques are best treated as promising until other groups report on them.

12.5.5 Validating Outputs

Every language model ranker produces text, and text must be turned back into a ranking of real candidates. A robust parser maps each output token or name to the candidate set, discards items that are not candidates, removes duplicates, and decides what to do about candidates the model omitted. Each of those events should be counted and logged per request, because their rates are early warnings of prompt drift, model changes, and inputs the model handles badly.

The choice of fallback for omitted candidates deserves particular care. The companion code's parser reads only the text after the `Ranking:` marker on its line, or the first non-empty line when the marker is missing, so that letters inside an explanation are not mistaken for an answer. It appends missing candidates in a seeded random order, because an order inherited from the slate can encode the answer. If the slate order is not random, or if the target item was placed at a fixed position, a model that fails to parse can still score well or badly depending on where the target happened to sit. A serving system can fall back to the upstream ranker's order, which is its best available guess, but an evaluation should never use an order that encodes the answer, and evaluation reports should include the fallback rate alongside the metric. Constrained decoding, as in Section 12.2.6, removes out-of-set items at the source when the serving stack allows it, and permutation-based bootstrapping, as in Hou et al. (2024), reduces position bias at the cost of several calls per request.

12.5.6 Where Language Model Rankers Fit in Serving

A listwise prompt with a short history and twenty candidates runs to several hundred tokens, and a reasoning model may add a thousand output tokens before its answer. Compared with a ranker that scores candidates with a few matrix multiplications, this is orders of magnitude more computation per request, and its latency grows with the length of the output. Language model rankers therefore fit best where the list is short and the time budget is generous. Final re-ranking of a few dozen items is one such place. Asynchronous surfaces such as email digests, push notifications, and pre-generated home-page modules are better, because their recommendations can be computed minutes or hours ahead, as Spotify does by triggering generation on user events (D'Amico et al., 2026). Where a model must run on the request path, the levers of Section 12.1 apply: distil the large model into a small one, as Instacart does for its long-tail queries (Zhu et al., 2025), cache results for frequent contexts, and remove written reasoning from the served policy, as the GR2 technical report does (Li et al., 2026).

12.6 Evaluating LLM-Based Recommenders

Chapter 3 laid out offline evaluation for recommenders, and Chapter 16 covers online experiments. Language model rankers add pitfalls of their own, mostly because their cost forces evaluations onto small candidate slates and small samples of users, and because the models arrive having read much of the public internet.

12.6.1 How the Slate Decides the Result

A language model ranker is usually evaluated on slates: for each test session, a small set of candidates, perhaps twenty, that contains the item the user actually engaged with next, called the target, together with negatives. How the negatives are chosen sets the difficulty of the task. With random negatives, the target is often the only candidate related to the user's history, and almost any sensible ranker finds it. With negatives retrieved as the nearest neighbours of the history, every candidate looks plausible and the task is much harder. Hou et al. (2024) showed that the ranking quality of language models varies

with the retriever that supplies the candidates, so two numbers measured on differently built slates are not comparable.

A subtler problem arises when the retriever misses the target. Many protocols then insert the target into the slate so that every session can be scored. The inserted target is, by construction, an item the retriever judged less similar to the history than every other candidate. A ranker that orders candidates by how well their text matches the history, which is roughly what a zero-shot language model does when it reads product titles, will push the inserted target toward the bottom. Figure 12.8 contrasts the two situations.



Figure 12.8: How slate construction changes the evaluation of a language model ranker. Left: with random negatives, a ranker that matches candidates to the history by text similarity finds the target easily. Right: with nearest-neighbour negatives and the target injected when retrieval missed it, the injected target is by construction less similar to the history than the other candidates, so the same ranker pushes the target down. A random ranker's expected score is the same in both slates.

The figure explains a result that otherwise looks absurd: a language model ranker scoring below a random ordering. A random ranker's expected metric depends only on the slate size, while a similarity-driven ranker is systematically misled on every slate with an injected target. A result below random under this protocol is a property of the protocol. Two other details can amplify the effect. If the injected target always sits at the same position in the prompt, the model's position bias and any parsing fallback that preserves input order (Section 12.5.5) both interact with that position. And if a fine-tuned ranker was trained with random negatives but is evaluated on nearest-neighbour slates, its training and evaluation distributions differ.

A sound protocol reports how often the target had to be injected, and reports ranking metrics separately on slates where the retriever found the target, which measure the ranker, and end-to-end recall over all sessions, which measures the pipeline. It shuffles candidate order independently for every slate, uses the same slates for every model being compared, matches the negatives used for fine-tuning to those used for evaluation, and includes the upstream ranker's own order and a popularity ranking as baselines alongside random.

12.6.2 Leakage

Leakage means that information unavailable at prediction time reaches the model, and language models introduce three new routes for it. The first is pre-training. Public recommendation datasets, product descriptions, and reviews are widely published, and a language model may have seen them during pre-training, so a strong zero-shot result on a public benchmark can partly reflect memorization. Evaluations on proprietary data, or on interactions that occurred after the model's training data was collected, are more trustworthy. The second route is generated features. A user summary written by a language model from the user's complete history, including the test period, leaks the answer into the features, which is the time-travel failure that Chapter 4 describes in Section 4.4; summaries must be regenerated from the history available at each prediction point. The third route is the encoder. If the item encoder behind a quantizer or a ranker was fine-tuned on co-engagement pairs that include test-period interactions, the test items' embeddings already reflect the behaviour being predicted, so encoders must be trained on the training period only. The chapter's companion code uses a base sentence encoder by default for this reason.

12.6.3 Baselines and Sample Sizes

Language model inference is expensive, so evaluations often run on a few hundred or a few thousand sessions where a conventional model would be scored on millions. Small samples make confidence intervals essential, and differences of a point or two in recall at a small cutoff are often within noise at these sizes; the statistical refresher in Section 16.4 covers confidence intervals. Every model in a comparison should be scored on the same sessions and the same slates, so that differences come from the models. The baselines should include a strong conventional model trained on the same data, such as a sequential recommender from Chapter 11, because a language model that beats only random and popularity baselines has not yet shown that it earns its cost.

12.6.4 Offline Metrics, Online Tests, and Language Model Judges

The published deployments in this chapter were validated by online experiments: Meta's Generative Recommenders (Zhai et al., 2024), Kuaishou's OneRec (OneRec Team, 2025), Spotify's GLIDE (D'Amico et al., 2026), and the KAR deployment at Huawei (Xi et al., 2024) all report A/B tests. Generative systems give at least two reasons not to rely on offline metrics alone. They change what users are shown, including its diversity and novelty, in ways that next-item accuracy does not capture, and their failures, such as an off-brand generated theme or a confidently wrong attribute, are rare events that small offline samples miss. Chapter 16 covers the design of these experiments.

Some qualities of generated output, such as whether a page theme is coherent or an explanation accurate, have no logged label, and teams increasingly prompt a language model as a judge to grade outputs against a rubric. Instacart, for example, runs language model judges on a small share of users to check theme quality and brand compliance (Hasani et al., 2026). Judges need the same scepticism as any other model. They can share blind spots with the system they evaluate, especially within one model family, and they are sensitive to the order and wording of what they are shown. An automated score also becomes a target once a team optimizes against it, as the GR2 reward hacking in Section 12.5.4 shows. Calibrate every judge against human labels on a sample, keep a human-reviewed holdout that is never used for tuning, and avoid using one judge as both a training signal and the final measure of success.

12.7 Choosing an Integration Pattern

The previous sections described five ways to bring generative models into a recommender: offline enrichment, Semantic IDs as ranking features, language model embeddings and generative retrieval, language model re-ranking with distilled students, and end-to-end generative ranking. No single pattern is best for every team. Two questions narrow the choice. How much added serving cost and latency can the product tolerate? And how much interaction data and modelling capacity, meaning engineers, training infrastructure, and GPUs, does the team have? Figure 12.9 places the patterns on those two axes.



Figure 12.9: Choosing an integration pattern by tolerance for added serving cost and latency (horizontal axis) and by interaction data and modelling capacity (vertical axis). Offline enrichment and Semantic IDs as features suit teams with little of either. Language model re-ranking, on asynchronous surfaces or through distilled students on interactive ones, suits products that can afford serving cost but have limited data. Language model embedding retrieval and generative retrieval suit data-rich teams that need cheap serving, and end-to-end generative ranking and retrieval suit teams with both.

The lower-left quadrant is where most teams should start. Offline enrichment adds no request-path cost, works with a catalog of any size, and improves features that an existing ranker already consumes, and Semantic IDs as ranking features change one feature family in a model the team already operates. Both are reversible, and both can be evaluated with the offline and online tooling the team already has.

Moving right, toward more serving cost, makes room for language model re-ranking. Asynchronous surfaces such as email digests and notification candidates are the natural first home for it, because their latency budget is measured in minutes. On interactive surfaces, a distilled student or a cached result usually has to stand in for the large model, as in the cascades of Section 12.4.

Moving up, toward more data and capacity, makes language model embeddings and generative retrieval worthwhile. These patterns need large volumes of interaction data to train well, but they can be served cheaply: embeddings are computed nearline and searched with nearest-neighbour indexes, as at LinkedIn (Danchev, 2026), and generative retrieval can be precomputed for each user or triggered by events, as at Spotify (D'Amico et al., 2026). The usual first deployment adds the new model as one more retrieval source alongside the existing ones, which limits the risk and makes the new source's contribution measurable.

The upper-right quadrant holds end-to-end generative ranking and retrieval in the style of HSTU and OneRec. The published examples come from platforms with very large logged histories, GPU serving fleets, and dedicated infrastructure teams (Zhai et al., 2024; OneRec Team, 2025). A team without those can still prepare for this quadrant by logging interactions as timestamped sequences with action types, by keeping rich item text and images, and by building the evaluation setup of Section 12.6, so that a later move upward does not start from scratch. Table 12.1 summarizes the trade-offs.

Table 12.1: Generative integration patterns compared.

Pattern	Where the model runs	Main prerequisites	Main risk
Offline enrichment (12.4)	Batch jobs before any request	Item text, a feature pipeline, human review of samples	Silent feature bugs: coverage gaps, bad joins, mismatched encoders
Semantic IDs as ranking features (12.2.8)	Quantizer offline; features served as usual	Content embeddings and an existing ranker	Codebook collapse and frequent collisions
Embedding or generative retrieval (12.2)	Nearline embeddings or precomputed generation	Large interaction logs and GPU training	New items never retrieved; invalid identifiers
Language model re-ranking and distilled students (12.5)	Short lists on the request path, or asynchronous surfaces	Preference labels and a latency budget	Position bias, invalid outputs, serving cost
End-to-end generative ranking (12.3)	Request path on GPUs	Long action sequences, GPU fleet, streaming training	Infrastructure cost and operational complexity

A team's position on the quadrants changes as it grows, so the choice is worth revisiting whenever one of the two axes moves.

12.8 Risks and Operations

Generative components fail in ways that conventional recommenders do not. This section covers hallucinated and stale items, prompt sensitivity, model licences, privacy, and cost, with the operational habits that contain each, and closes with agents that recommend to users at serving time.

12.8.1 Hallucinated and Stale Items

A hallucinated item is an output that refers to something outside the catalog: a Semantic ID that maps to no item, a product name that matches nothing, or an attribute that the product does not have. Section 12.2 showed how constrained decoding over a prefix tree removes invalid identifiers at the source, and Section 12.5.5 showed how a parser should map free-text outputs back to the candidate set. Two further cases need attention. Generated content can be correct when written and wrong later, because the catalog changes, so every generated artefact should carry the catalog version it was built

from, and serving should filter generated items against current availability, as it does for any other retrieval source. The second case is the plausible wrong fact, such as an incorrect material or a wrong compatibility claim, which no parser can catch. Sampling generated attributes for human review, and comparing them against structured catalog data where it exists, is the only reliable defence.

12.8.2 Prompt Sensitivity and Versioning

Language model outputs can change substantially with small changes to a prompt's wording, the order of its examples, or the order of candidates, and Hou et al. (2024) documented the position bias that candidate order alone introduces. Hosted models also change underneath their users when providers update or retire them. Treat prompts as code: keep them in version control, record the prompt version and the model identifier with every output, and run a fixed regression set of inputs through any changed prompt or model before it reaches production, comparing the outputs with the previous version's. Pin hosted model versions where the provider allows it, and plan for the day each pinned version is retired.

12.8.3 Model Licences

Open weights mean that the weights can be downloaded, and the licence decides what may be done with them. Some open-weight models are released under permissive licences that allow commercial use; others restrict commercial use, cap the size of the business that may use them, or impose acceptable-use conditions. Licences can differ even within one model family. For example, the 3-billion-parameter instruction-tuned release of Qwen2.5 is distributed under the Qwen Research License (Qwen Team, 2024), so a team that prototypes with it must confirm that its intended use is permitted before shipping a product built on it, which is one reason the chapter's companion code defaults to other sizes of the family. Fine-tuned derivatives generally inherit the base model's terms, and the terms of hosted model providers may restrict using their outputs to train other models, which matters for the distillation patterns of Section 12.4. Record the licence of every model and dataset in the model registry, and have it reviewed before a prototype becomes a product.

12.8.4 Privacy

A user's history in a prompt is personal data. Sending it to a third-party model provider requires a data processing agreement, a clear understanding of how long the provider retains it, and compliance with the regulations of every region the product serves. Minimize what each prompt contains: item titles and categories are usually enough, and user identifiers, contact details, and free-text messages are rarely needed. Generated user summaries are personal data too, so they must be covered by deletion requests along with the logs they were generated from, and so must any fine-tuned model trained on them, which in practice means keeping the ability to retrain without a deleted user's data. Self-hosting an open-weight model keeps data inside the organization at the cost of running inference infrastructure. Design choices about inputs also carry fairness consequences. LinkedIn, for example, states that its feed models rely on professional signals and engagement patterns and never on demographic attributes, and that its ranking model is audited for equitable treatment (Danchev, 2026).

12.8.5 Cost Monitoring

The cost of a generative component is driven by tokens and by GPU time, and both can drift without any change to the code. Longer user histories lengthen prompts, a new prompt version can double the output length, and a falling cache hit rate can multiply calls to the large model. Monitor input and output tokens per request, cache hit rates, the share of traffic that reaches each tier of a cascade, the

rates of invalid outputs and parsing fallbacks, GPU utilization, latency percentiles, and cost per thousand requests, and alert on sustained changes in each. Chapter 15 covers the monitoring stack these signals belong in, and Chapter 14 covers the latency side.

12.8.6 Agents That Recommend

An agent is a language model that pursues a goal over several steps: it plans, calls tools, observes their results, keeps a memory of the interaction, and decides when it is done. A serving-time recommendation agent talks with a user, works out what the user wants, and uses tools to find and rank items. In InteRecAgent, a language model acts as the controller and conventional recommender models act as its tools: an information-query tool answers questions about items, retrieval tools find candidates by hard conditions, such as a price limit, and by soft conditions, such as similarity to named items, and a ranking tool orders candidates with a recommendation model that uses the user's profile (Huang et al., 2023). A shared candidate bus passes items between tools without placing long lists in the prompt, the agent writes its whole plan before executing it, and a second language model acts as a critic that triggers a retry when results look wrong. The authors report that the design outperformed general-purpose language models as a conversational recommender on public datasets. RecMind plans tool calls more autonomously, and its authors report performance comparable to a fully trained recommendation model on several tasks without task-specific training (Wang et al., 2024c).

Agents multiply the risks of this section. Each turn may involve several model and tool calls, so latency and cost grow with the plan, and every tool call needs timeouts and fallbacks. Hallucinated items, privacy, and prompt sensitivity apply at every step, and evaluation must cover whole conversations, usually with simulated users, with every plan and tool call logged so that failures can be traced. The design choice that limits these risks is InteRecAgent's: keep conventional recommender models as the tools that retrieve and rank, so that recommendation quality still comes from models trained on behaviour, and let the language model handle conversation and planning. Agents that automate recommender research and experimentation are the subject of Appendix C.

12.9 Summary

Generative models can enter a recommender at every stage of the funnel and before it. Each placement inherits the latency and cost budget of its stage, and the published systems that put large models near the request path contain that cost with the same levers: work moved offline or nearline, caching, distillation into small models, computation shared across the candidates of a request, and expensive models confined to short lists or asynchronous surfaces.

Semantic IDs are coarse-to-fine codes learned by residual quantization of content embeddings. Building them well means watching codebook health, initializing codebooks with k-means, resetting dead codes, and separating collisions, which a suffix token fixes once, from invalid generations, which constrained beam search over the catalog's prefix tree prevents. Semantic IDs give new items an address without giving them traffic, so cold-start retrieval still needs prefix matching, a dense retriever, or an exploration path. Used as ranking features, they are often the cheapest way to benefit from them.

Meta's HSTU recasts ranking as predicting the next action in a sequence of items and actions, replaces softmax attention with pointwise attention and a relative bias for position and time, and keeps serving affordable by encoding a user's history once for many candidates. The industrial models built on related ideas run on infrastructure that few organizations can match, and the lessons that transfer to smaller teams are to log rich interaction sequences and to adopt shared-history scoring.

Language models used offline need the discipline of any feature pipeline, especially matched encoder spaces, complete coverage, and stable joins. As rankers they are most cost-effective with listwise prompts, improve sharply with fine-tuning that masks labels correctly, and in their reasoning form need rewards that copying the input order cannot satisfy. Their evaluation depends on how candidate slates are built, on leakage through pre-training and generated features, and on baselines strong enough to justify their cost.

A team chooses among these patterns by its tolerance for serving cost and by its data and modelling capacity, and most should start with offline enrichment and Semantic ID features. Whatever the pattern, hallucinated items, prompt changes, licences, privacy, and cost need explicit owners and monitoring, and conversational agents multiply each of those risks. Chapter 13 turns to the deployment practices that carry any of these models, generative or not, into production.

12.10 Exercises

The exercises cover each of the chapter's main patterns. Several build on the chapter's companion code in the book's repository; they ask you to design a change or a diagnostic and to reason about what it should show, and they leave the results to you.

Exercise 1: Codebook Health Check

The RQ-VAE in the companion code reports, for each quantization level, the number of codewords in use, the perplexity of the code assignments, and the number of distinct prefixes, and, for the whole catalog, the share of items that share their codes with another item and the size of the largest such group. Extend these statistics with the size of the largest group of items under a single first-level code, and show how perplexity relates to the entropy of the code distribution. The training script can switch off k-means initialization and the dead-code reset independently, which gives four combinations. Before training any of them, write down which statistics each remedy should move and in which direction; then train all four and compare the statistics with your predictions. Explain how you would tell whether a change in the collision share comes from better codebook use or from a change in the number of levels or the codebook size.

Exercise 2: Constrained Beam Search

The companion code's TIGER-style retriever already constrains beam search with a prefix tree of the catalog's Semantic IDs, so every beam it returns names a real item. Add an option that removes the constraint, so that each step extends beams with any code of the current level. Extend the existing beam-search tests to the new option, confirming that the constrained path still returns only real items and that the unconstrained path can return sequences that name no item, and design an evaluation that reports the invalid-sequence rate with and without the constraint, separately from collision handling. Explain why widening the beam and filtering, the alternative used by Rajput et al. (2023), can still return fewer items than requested.

Exercise 3: Shared-History Scoring

Using the companion HSTU-style encoder, design an attention mask that lets a microbatch of candidates appended after a user's history attend to the history and to themselves, never to each other. Because the encoder fuses each item with an action, give every appended candidate a placeholder action, and define each candidate's score from the encoder's output at that candidate's position. Write a test that checks that each candidate's score in the microbatch equals its score when it is appended alone, and

work out which parts of the relative attention bias and of the attention normalization, if any, must also be adjusted for the test to pass. Then express, as a function of history length and microbatch size, how much attention computation the design saves compared with one encoder pass per candidate.

Exercise 4: Slate Audit

An evaluation script builds twenty-candidate slates from the nearest neighbours of each session's history and inserts the target whenever the retriever missed it. Instrument the script to record, for each slate, whether the target was inserted and where it appears in the prompt. Using Section 12.6.1, explain how a ranker that matches candidates to the history by text similarity will behave on slates with an inserted target, and derive the expected score of a random ranker on the same slates. Redesign the protocol so that it reports ranking quality and end-to-end recall separately, and list the baselines the redesigned report should include.

Exercise 5: Label Masking Test

Write a unit test for a fine-tuning data pipeline that formats a prompt and answer pair with the model's chat template, applies right padding, and masks the labels. The test should decode the tokens whose labels are not -100 and assert that they consist only of the answer and the end-of-sequence token. Extend the test to left padding and show how a mask computed by slicing from the start of the sequence fails it. Explain why neither failure would be visible in the training loss.

Exercise 6: Position-Bias Probe

Design an experiment that measures the position bias of a zero-shot listwise language model ranker. For a fixed set of slates, present each slate in several random permutations, record the rank the model assigns to each candidate in each permutation, and define a statistic that separates the effect of a candidate's position from the effect of the candidate itself. Then design a bootstrapped aggregation in the style of Hou et al. (2024) for this ranker, and work out how many model calls per request it adds and how you would decide whether the reduction in bias is worth them.

Exercise 7: Encoder-Space Diagnostic

A ranker uses item vectors from a fine-tuned sentence encoder and session-intent vectors produced by encoding language model summaries with the base version of the same encoder, and one of its features is the cosine similarity between the two. Design a diagnostic that tests whether this feature is meaningful, for example by comparing the similarity distribution for sessions paired with the items they went on to engage with against sessions paired with random items, computed once with the mismatched encoders and once with a single encoder. Describe what you would expect to see if the two spaces are incompatible, and propose two fixes with their trade-offs.

Exercise 8: Enrichment Cascade Cost Model

Design a three-tier cascade for extracting product attributes from a catalog: rules, then a small fine-tuned model, then a large language model. Write a cost model in which each tier has a cost per item, a fraction of the items reaching it that it resolves confidently, and an accuracy on the items it resolves, and derive the expected cost and accuracy of the whole cascade. Explain how you would choose the confidence thresholds that route items between tiers, what you would monitor once the cascade runs, and how the large model's outputs would be used to retrain the small model over time.

References

Bao, K., Zhang, J., Zhang, Y., Wang, W., Feng, F., and He, X. (2023). "TALLRec: An Effective and Efficient Tuning Framework to Align Large Language Model with Recommendation." Proceedings of the 17th ACM Conference on Recommender Systems. ACM. https://doi.org/10.1145/3604915.3608857

D'Amico, E., De Nadai, M., Chandar, P., Vohra, D., Lin, S., Lefarov, M., Gigioli, P., Penha, G., et al. (2026). "Deploying Semantic ID-based Generative Retrieval for Large-Scale Podcast Discovery at Spotify." arXiv preprint arXiv:2603.17540. https://arxiv.org/abs/2603.17540

Dai, S., Shao, N., Zhao, H., Yu, W., Si, Z., Xu, C., Sun, Z., Zhang, X., and Xu, J. (2023). "Uncovering ChatGPT's Capabilities in Recommender Systems." Proceedings of the 17th ACM Conference on Recommender Systems. ACM. https://doi.org/10.1145/3604915.3610646

Danchev, H. (2026). "Engineering the Next Generation of LinkedIn's Feed." LinkedIn Engineering Blog. https://www.linkedin.com/blog/engineering/feed/engineering-the-next-generation-of-linkedins-feed

De Cao, N., Izacard, G., Riedel, S., and Petroni, F. (2021). "Autoregressive Entity Retrieval." International Conference on Learning Representations (ICLR 2021). https://arxiv.org/abs/2010.00904

Dettmers, T., Pagnoni, A., Holtzman, A., and Zettlemoyer, L. (2023). "QLoRA: Efficient Finetuning of Quantized LLMs." Advances in Neural Information Processing Systems 36 (NeurIPS 2023). https://arxiv.org/abs/2305.14314

Ding, Q., Course, K., Ma, L., Sun, J., Liu, R., Zhu, Z., Yin, C., Li, W., Li, D., Shi, Y., et al. (2026). "Bending the Scaling Law Curve in Large-Scale Recommendation Systems." arXiv preprint arXiv:2602.16986. https://arxiv.org/abs/2602.16986

Hasani, M., Shahidi, H., Levinson, T., and Shu, G. (2026). "Our Early Journey to Transform Instacart's Discovery Recommendations with LLMs." Instacart Tech Blog. https://company.instacart.com/tech-innovation/our-early-journey-to-transform-instacart-s-discovery-recommendations-with-llms

He, R., Heldt, L., Hong, L., Keshavan, R., Mao, S., Mehta, N., Su, Z., Tsai, A., Wang, Y., Wang, S.-C., Yi, X., et al. (2025). "PLUM: Adapting Pre-trained Language Models for Industrial-scale Generative Recommendations." arXiv preprint arXiv:2510.07784. https://arxiv.org/abs/2510.07784

Hou, Y., Zhang, J., Lin, Z., Lu, H., Xie, R., McAuley, J., and Zhao, W. X. (2024). "Large Language Models are Zero-Shot Rankers for Recommender Systems." Advances in Information Retrieval: 46th European Conference on Information Retrieval (ECIR 2024). Springer. https://arxiv.org/abs/2305.08845

Hsiao, K.-J., Feng, Y., and Lamkhede, S. (2025). "Foundation Model for Personalized Recommendation." Netflix Technology Blog. https://netflixtechblog.com/foundation-model-for-personalized-recommendation-1a0bd8e02d39

Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., and Chen, W. (2022). "LoRA: Low-Rank Adaptation of Large Language Models." International Conference on Learning Representations (ICLR 2022). https://arxiv.org/abs/2106.09685

Huang, X., Lian, J., Lei, Y., Yao, J., Lian, D., and Xie, X. (2023). "Recommender AI Agent: Integrating Large Language Models for Interactive Recommendations." arXiv preprint arXiv:2308.16505. https://arxiv.org/abs/2308.16505

Kang, W.-C., and McAuley, J. (2018). "Self-Attentive Sequential Recommendation." 2018 IEEE International Conference on Data Mining (ICDM). IEEE. https://arxiv.org/abs/1808.09781

Lee, D., Kim, C., Kim, S., Cho, M., and Han, W.-S. (2022). "Autoregressive Image Generation using Residual Quantization." Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR 2022). https://arxiv.org/abs/2203.01941

Li, H., Liu, X., Nie, J., Wen, E., Yang, C., Yang, J., Yu, N., Beg, H., Arditi, G., and Bhatia, N. (2025). "Meta's Generative Ads Model (GEM): The Central Brain Accelerating Ads Recommendation AI Innovation." Engineering at Meta.

https://engineering.fb.com/2025/11/10/ml-applications/metas-generative-ads-model-gem-the-central-brain-accelerating-ads-recommendation-ai-innovation/

Li, Y., Zhang, Z., Liang, M., Asadi, K., Xu, J., Kim, J., Bai, C., Zhang, J., et al. (2026). "GR2 Technical Report." arXiv preprint arXiv:2606.31984. https://arxiv.org/abs/2606.31984

Liang, M., Li, Y., Xu, J., Asadi, K., Liu, X., Gu, S., Rangadurai, K., Shyu, F., et al. (2026). "GR2: Generative Reasoning Re-ranker." arXiv preprint arXiv:2602.07774. https://arxiv.org/abs/2602.07774

OneRec Team. (2025). "OneRec Technical Report." arXiv preprint arXiv:2506.13695. https://arxiv.org/abs/2506.13695

Qwen Team. (2024). "Qwen2.5-3B-Instruct." Model card and licence, Hugging Face. https://huggingface.co/Qwen/Qwen2.5-3B-Instruct

Rajput, S., Mehta, N., Singh, A., Keshavan, R., Vu, T., Heldt, L., Hong, L., Tay, Y., Tran, V. Q., Samost, J., Kula, M., Chi, E. H., and Sathiamoorthy, M. (2023). "Recommender Systems with Generative Retrieval." Advances in Neural Information Processing Systems 36 (NeurIPS 2023). https://arxiv.org/abs/2305.05065

Reimers, N., and Gurevych, I. (2019). "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks." Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP-IJCNLP). ACL. https://arxiv.org/abs/1908.10084

Singh, A., Vu, T., Mehta, N., Keshavan, R., Sathiamoorthy, M., Zheng, Y., Hong, L., Heldt, L., Wei, L., Tandon, D., Chi, E. H., and Yi, X. (2024). "Better Generalization with Semantic IDs: A Case Study in Ranking for Recommendations." Proceedings of the 18th ACM Conference on Recommender Systems. ACM. https://doi.org/10.1145/3640457.3688190

Van den Oord, A., Vinyals, O., and Kavukcuoglu, K. (2017). "Neural Discrete Representation Learning." Advances in Neural Information Processing Systems 30 (NIPS 2017). https://arxiv.org/abs/1711.00937

Wang, H., Narayanan S, M., Gungor, O., Xu, Y., Kamath, K., Chalasani, R., Hazra, K. S., and Rao, J. (2024a). "Improving Pinterest Search Relevance Using Large Language Models." 3rd International Workshop on Industrial Recommendation Systems at CIKM 2024. https://arxiv.org/abs/2410.17152

Wang, W., Bao, H., Lin, X., Zhang, J., Li, Y., Feng, F., Ng, S.-K., and Chua, T.-S. (2024b). "Learnable Item Tokenization for Generative Recommendation." Proceedings of the 33rd ACM International Conference on Information and Knowledge Management (CIKM 2024). ACM. https://doi.org/10.1145/3627673.3679569

Wang, Y., Jiang, Z., Chen, Z., Yang, F., Zhou, Y., Cho, E., Fan, X., Huang, X., Lu, Y., and Yang, Y. (2024c). "RecMind: Large Language Model Powered Agent For Recommendation." Findings of the Association for Computational Linguistics: NAACL 2024. ACL. https://arxiv.org/abs/2308.14296

Xi, Y., Liu, W., Lin, J., Cai, X., Zhu, H., Zhu, J., Chen, B., Tang, R., Zhang, W., et al. (2024). "Towards Open-World Recommendation with Knowledge Augmentation from Large Language Models." Proceedings of the 18th ACM Conference on Recommender Systems. ACM. https://doi.org/10.1145/3640457.3688104

Xu, S., and Vadrevu, S. C. (2024). "Building DoorDash's Product Knowledge Graph with Large Language Models." DoorDash Engineering Blog. https://careersatdoordash.com/blog/building-doordashs-product-knowledge-graph-with-large-language-models/

Yang, L., Paischer, F., Hassani, K., Li, J., Shao, S., Li, Z. G., He, Y., Feng, X., Noorshams, N., Park, S., Long, B., Nowak, R. D., Gao, X., and Eghbalzadeh, H. (2024). "Unifying Generative and Dense Retrieval for Sequential Recommendation." arXiv preprint arXiv:2411.18814. https://arxiv.org/abs/2411.18814

Zeghidour, N., Luebs, A., Omran, A., Skoglund, J., and Tagliasacchi, M. (2021). "SoundStream: An End-to-End Neural Audio Codec." arXiv preprint arXiv:2107.03312. https://arxiv.org/abs/2107.03312

Zhai, J., Liao, L., Liu, X., Wang, Y., Li, R., Cao, X., Gao, L., Gong, Z., Gu, F., He, M., Lu, Y., and Shi, Y. (2024). "Actions Speak Louder than Words: Trillion-Parameter Sequential Transducers for Generative Recommendations." Proceedings of the 41st International Conference on Machine Learning (ICML 2024). PMLR. https://arxiv.org/abs/2402.17152

Zhang, Z., Ni, K., Chen, Y., Liang, M., Wei, X., Pu, Y., Tian, F., Sun, C., et al. (2026). "Diffusion-GR2: Diffusion Generative Reasoning Re-ranker." arXiv preprint arXiv:2607.01170. https://arxiv.org/abs/2607.01170

Zhu, Y., Shu, G., Fan, R., Gudla, V., and Tenneti, T. (2025). "Building the Intent Engine: How Instacart is Revamping Query Understanding with LLMs." Instacart Tech Blog. https://company.instacart.com/tech-innovation/building-the-intent-engine-how-instacart-is-revamping-query-understanding-with-llms